Lenguaje Python

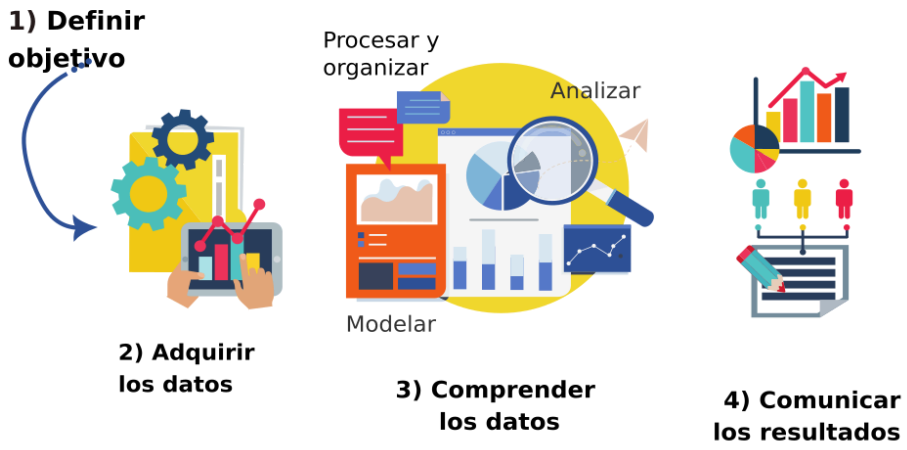
Cursada 2025

Introducción al análisis de datos

- pandas
- Matplotlib

Ciencia de datos:

> En pocas palabras: se refiere al análisis significativo de datos aplicando técnicas de programación sobre un área de conocimiento específico.



¿Cómo podemos representar datos?

- Mapa de cortes de electricidad.
- Mapa poblacional con información de los censos en Argentina.
- Mapa provincial sobre resultados de elecciones.

Manejo de archivo de datos

- ¿Cuáles son las ventajas de utilizar pandas sobre las estructuras de datos estándar de Python?

- ¿Cómo puede simplificar las operaciones realizadas con los módulos **csv** y **json**?

- ¿Qué otros formatos de archivos conocen para almacenar datos?
- ¿Cuáles son formatos abiertos?

Introducción a pandas

- ¿Qué es pandas y cuál es su importancia en el análisis de datos?
- ¿Cuáles son las principales estructuras de datos en pandas?
- ¿Cómo importar y explorar un conjunto de datos usando pandas?
- ¿Qué funcionalidades ofrece pandas para conocer, filtrar y modificar los datos?

🎒 pandas

- Es una biblioteca de Python que proporciona estructuras de datos y herramientas de análisis de datos de alto rendimiento y fáciles de usar.
- Esta orientada al análisis de datos porque permite manipular y analizar conjuntos de datos de manera eficiente.

In [ ]:
```
import pandas as....
```

In [ ]:
```
import pandas as pd
```

📌 Tipo de archivos y estructuras de datos en pandas

¿Qué tipo de datos usaron con para recorrer el contenido usando:

- csv
- json

Veamos un ejemplo

In [ ]:
```
from pathlib import  Path
file_route = Path('ejemplos')/'clase09'
file_data = 'ar-airports.csv'
df_airports = pd.read_csv(file_route/file_data)
```

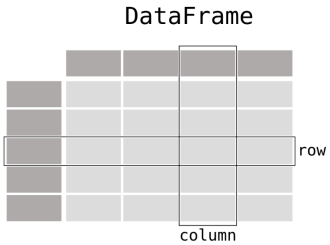
In [ ]:
```
df_airports
```

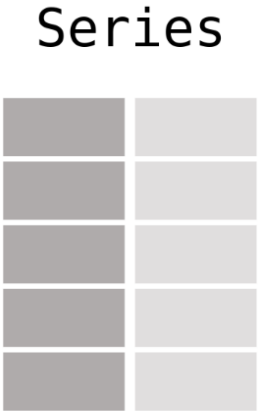
¿De qué tipo de datos es la variable utilizada?

In [ ]:
```
type(df_airports)
```

Las principales estructuras de datos en pandas

- **Series**: es un arreglo unidimensional de datos etiquetados.
- **DataFrame**: es una estructura de datos tabular bidimensional compuesta por filas y columnas.





Tipo de archivos y formas de abrir:

- **pd.read_csv()**: para leer archivos CSV (Comma Separated Values).
- **pd.read_excel()**: para leer archivos de Excel.
- **pd.read_json()**: para leer archivos JSON.
- **pd.read_html()**: para leer tablas HTML de una página web.
- **pd.read_sql()**: para leer datos de una base de datos SQL.
- **pd.read_hdf()**: para leer archivos HDF5 (Hierarchical Data Format versión 5).
- Muchos más
- Más info sobre opciones para lectura de distintos tipos de archivos:

> Capítulo 6 "Python for Data Analysis"

📌 Conociendo nuestros datos

Ver las primeras filas o últimas

```
df.head(number)
df.tail(number)
```

In [ ]:
```
df_airports.head(5)
```

Filas y columnas

In [ ]:
```
df_airports.shape
```

Nombres de las columnas

Si trabajamos con un archivo csv, ¿en dónde se encuentran los nombres de las columnas?

In [ ]:
```
df_airports.columns
```

Información de nuestros datos

- ¿Qué otra información nos puede servir, por ejemplo si vamos a buscar datos en la columna **municipality** del archivo de aeropuertos?
- ¿Qué problemas tiene esta columna?

In [ ]:
```
df_airports.municipality
```

En principio no se se detecta un problema, pero utilicemos algunas funcionalidades para conocer los datos, antes de poder hacer un análisis de datos

In [ ]:
```
df_airports.info()
```

¿Cómo podemos obtener los valores de algunos cálculos estadísticos básicos, sobre las columnas con datos numéricos?

In [ ]:
```
df_airports.describe()
```

¿Qué es cada cálculo?

- **count**: cantidad de valores no nulos
- **mean**: media, el valor promedio de un conjunto de datos
- **std**: desviación estándar, cuánto varían los valores en relación con la media.
- **min**: mínimo
- **25%**(primer cuartil): indica el valor por debajo del cual cae el 25% de los datos.
- **50%**(mediana): es el valor que divide la serie en dos mitades iguales.
- **75%**(tercer cuartil): indica el valor por debajo del cual cae el 75% de los datos.
- **max**: máximo

In [ ]:
```
df_airports.dtypes
```

- **info()**: cantidad de valores nulos por columnas y tipo de datos que pandas le asignó.
- **describe()**: cálculos sobre las columnas que contienen datos numéricos.
- **dtypes**: solamente los tipos de datos asignados por pandas.
  - [Info tipos de datos](https://pandas.pydata.org/pandas-docs/stable/user_guide/basics.html#basics-dtypes

- Otras operaciones estadísticas :
  - min, max
  - argmin, argmax
  - idxmin, idxmax
  - quantile
  - sum
  - mean
  - median
  - mad: desviación absoluta media respecto al valor medio
  - prod: producto de todos los valores
  - var: varianza muestral de los valores

- std: desviación estándar muestral de los valores
- skew: asimetría muestral
- kurt: curtosis muestral de los valores
- cumsum: suma acumulativa de los valores
- cummin, cummax: mínimo o máximo acumulativo de los valores
- cumprod: calcular la primera diferencia aritmética
- pct_change: calcular cambios porcentuales

> Capítulo 5.3 "Python for Data Analysis"

¿Cuánta memoria estamos usando con nuestro dataset?

In [ ]:
```
file_ind = 'usu_individual_T324.txt'
```

In [ ]:
```
usi_ind = pd.read_csv(file_route/file_ind, delimiter=';')
```

El error :

> DtypeWarning: Columns (102) have mixed types. Specify dtype option on import or set low_memory=False

- **pandas** indica que la columna 102 del archivo contiene datos de diferentes tipos (por ejemplo, números y cadenas de texto). Esto puede causar problemas al procesar los datos, ya que pandas no sabe cómo manejar esos tipos mixtos.
- Se puede usar **low_memory=False**: pandas procesa el archivo sin advertencias.

In [ ]:
```
usi_ind = pd.read_csv(file_route/file_ind, delimiter=';', low_memory=Fals
```

In [ ]:
```
usi_ind.info(verbose=False, memory_usage='deep')
#df_airports.info(verbose=False, memory_usage='deep')
```

👉 Quiero saber la cantidad de aeropuertos que hay de cada tipo

1. Consulto los nombres de las columnas para saber donde está esa información

In [ ]:
```
df_airports.columns
```

2. Consulto cuáles son los valores únicos de la columna **type**

In [ ]:
```
df_airports.type.unique()
```

3. Calculo cuántos hay de cada tipo

In [ ]:
```
df_airports['type'].value_counts()
```

📌 Trabajando con datos de una sola columna

- Queremos guardar los nombres de todos los aeropuertos

In [ ]:
```
df_names = df_airports.name
```

In [ ]:
```
df_names
```

¿Cómo puedo obtener los nombres en una estructura más simple?

In [ ]:
```
df_airports.name.to_list()[:5]
```

In [ ]:
```
df_airports.name.values[:5]
```

```
df_airports.name.to_list()
```

- Convierte la columna en una lista de Python.
- **Tipo de Objeto Devuelto**: devuelve una lista de Python (list).
- **Uso**: es útil cuando se necesita trabajar con los datos como una lista, ya que las listas son una estructura de datos nativa de Python y son fáciles de manipular.

```
df_airports.name.values
```

- Devuelve los valores de la serie como un array de NumPy.
- **Tipo de Objeto Devuelto**: devuelve un array de NumPy (numpy.ndarray).
- **Uso**: es útil cuando se necesita realizar operaciones numéricas o de matriz, ya que NumPy proporciona una amplia gama de funciones y métodos optimizados para trabajar con arrays.
- Veremos más adelante qué características tiene la librería NumPy y las facilidades que provee.

¿Pero es lo mismo un Dataframe que una variable de tipo Series?

📌 Series

- Tipo de dato unidimensional, es como un arreglo con etiquetas de índice.
- Los tipos de operaciones pueden variar con respecto a un Dataframe.
- Info

In [ ]:
```
df_airports.describe()
```

In [ ]:
```
type(df_airports.elevation_ft)
```

In [ ]:
```
df_airports.elevation_ft.describe()
```

1. Las estadísticas se muestran de todas las columnas númericas.

2. Las estadísticas se muestran sólo de la columna seleccionada.

La estructura **Series** tiene dos componentes principales: índice y valores:

- **Índice (index)** es una etiqueta que se utiliza para **identificar** cada uno de los elementos de la Serie.
  - Por defecto, si no se especifica un índice al crear la Serie, pandas asigna un índice numérico que comienza en 0 y se incrementa en 1 para cada elemento.
  - **index**: permite acceder a los valores.
- **Valores (values)**: son los datos que contiene.
  - **values**: permite acceder a los valores.

In [ ]:
```
df_airports.elevation_ft.index
```

In [ ]:
```
df_airports.elevation_ft.values[:5]
```

📌 Acceso a datos por índice o etiquetas

In [ ]:
```
df_airports.iloc[10:14]
```

In [ ]:
```
df_airports.loc[0]
```

- **loc**: por label, en este caso el índice en un número
- **iloc**: por índice entero, es el número de la fila.
- Más info de acceso indexado en:

> Capítulo 5.2 "Python for Data Analysis"

🎒 Manipulación de Datos con pandas

- ¿Cómo seleccionar, filtrar y transformar datos?
- ¿Cuáles son las operaciones comunes para manipular y procesar datos?

📌 Filtrado de datos

- Filas que coincidan con un valor específico de una columna.
- Filas que cumplan condición con operadores booleanos.
- Selección de columnas específicas.
- Filas que coincidan con algún string o elemento de una lista.

👉 Queremos encontrar:

- los aeropuertos que coincidan con un tipo de aeropuerto dado.
- los aeropuertos que se encuentren en valores o rangos correspondientes a la elevación.
- los aeropuertos de tipo: large, medium, small.

👉 Los aeropuertos que coincidan con un tipo de aeropuerto dado.

In [ ]:
```
df_airports[df_airports.type=='medium_airport'].head(3)
```

¿Por qué usamos dos veces la variable **df_airports**?

In [ ]:
```
df_airports.type=='medium_airport'
```

- Nos devuelve un array de booleanos, es en realidad una máscara.
- Luego se aplica esa máscara a las filas del Dataframe.

In [ ]:
```
df_airports.type.unique()
```

In [ ]:
```
mask = df_airports.type=='medium_airport'
df_airports[mask].head(3)
```

👉 Los aeropuertos que se encuentren en valores o rangos correspondientes a la elevación.

In [ ]:
```
df_airports[df_airports.elevation_ft<131].head(3)
```

In [ ]:
```
df_airports[(df_airports.elevation_ft>131) & (df_airports.elevation_ft<90
```

¿Son los mismos operadores booleanos usados en Python?

📌 Operadores booleanos

Operador	Python	pandas
and	cond1 and cond2	(cond1) & (cond2)
or	cond1 or cond2	(cond1) | (cond2)
not	not cond	~cond

Cuando filtramos por valores numéricos podemos utilizar **between** en lugar de poner explícitamente ambas condiciones

In [ ]:
```
df_airports[df_airports.elevation_ft.between(131, 903)]
```

👉 Filtrado de los aeropuertos de tipo: large, medium, small

In [ ]:
```
df_airports.type.unique()
```

Hay dos posibles formas

- Filtrar los aeropuertos que contengan el string **airport**, ya que los demás no contienen ese string.
- Filtrar con una lista que contenga las categorías que estamos buscando.

👉1.Filtrar indicando el string **airport**, ya que los demás no lo contienen.

In [ ]:
```
df_airports[df_airports.type.str.contains('airport')].head(3)
```

`df.column.str.contains('string')`
Permite filtrar las filas que en la **columna** se encuentre el string dado

👉2. Filtrar con una lista que contenga las categorías que estamos buscando.

In [ ]:
```
airports_int = ['large_airport', 'medium_airport',  'small_airport']
```

In [ ]:
```
df_airports[df_airports.type.isin(airports_int)]
```

`df.columna.isin(lista)`
Permite filtrar las filas cuyo contenido de la **columna** dada sea algunos de los elementos de la **lista** pasada como parámetro.

📊 Gráficos con Matplotlib

¿Qué tipos de gráficos conocen?

Con Matplotlib se pueden realizar

- Barra
- Línea
- Torta
- Scatter
- Histograma
- Boxplot
- 3D
- y muchos más

Primero importamos:

In [ ]:
```
import matplotlib.pyplot as plt
```

Generar gráficos simples

Graficar los tipos de aeropuertos según el tipo

- barra
- torta

Utilizando plot directamente con los valores contados

In [ ]:
```
conteo_tipos = df_airports_int.type.value_counts()
conteo_tipos
```

In [ ]:
```
type(conteo_tipos)
```

Crear el gráfico de barra

In [ ]:
```
conteo_tipos.plot(kind='bar', xlabel='Tipo de aeropuerto', ylabel='Cantid
plt.show()
```

Crear el gráfico de torta

In [ ]:
```
conteo_tipos.plot(kind='pie', autopct='%1.1f%%', title='Proporción de aer

# Mostrar el gráfico de torta
plt.show()
```

📊 Utilizando las funciones de Matplotlib

In [ ]:
```
values = df_airports_int.type.value_counts().values
labels = df_airports_int.type.value_counts().index
```

📌 Crear el gráfico de barras

In [ ]:
```
plt.bar(labels,values)
# Agregar etiquetas y título
plt.xlabel('Tipo de aeropuerto')
plt.ylabel('Cantidad')
plt.title('Cantidad de aeropuertos por tipo')

# Rotar etiquetas en el eje x para mejor legibilidad
plt.xticks(rotation=45)
# Mostrar el gráfico de barras
plt.show()
```

📌Crear el gráfico de torta

In [ ]:
```
plt.pie(values, labels=labels, autopct='%1.2f%%')

# Agregar título
plt.title('Proporción de aeropuertos por tipo')

# Mostrar el gráfico de torta
plt.show()
```

- autopct: permite mostrar el porcentaje al lado de cada porción

📝Cambio de los colores del gráficos

In [ ]:
```
plt.pie(values, labels=labels, colors=['red', 'green', 'blue'])
plt.title('Proporción de aeropuertos por tipo')

plt.show()
```

📝Agregar sombras

In [ ]:
```
plt.pie(values, labels=labels, shadow=True)
plt.title('Proporción de aeropuertos por tipo')

plt.show()
```

📝 Separar una porción para remarcar

In [ ]:
```
explode = (0.1, 0, 0)  # Separar la segunda porción (índice 1)
plt.pie(values, labels=labels, explode=explode)
plt.title('Proporción de aeropuertos por tipo')

plt.show()
```

In [ ]:

In [ ]:
```
explode = (0.1, 0, 0)  # Separar la segunda porción (índice 1)
plt.pie(values, labels=labels, explode=explode)
plt.title('Proporción de aeropuertos por tipo')
plt.legend(labels, loc='center left', bbox_to_anchor=(1.5, 0.5))
plt.show()
```

Para pensar en casa

- ¿Cómo manejar valores faltantes y duplicados en un conjunto de datos?
- ¿Cómo agregamos columnas, y cómo podemos hacerlo aplicando una función?
- ¿Qué técnicas ofrece pandas para realizar agregaciones y agrupaciones de datos?
- Nuestros datos y gráficos interactuando con Streamlit.

Seguimos la próxima ...

Ejemplos extras

Modificamos los nombres de las columnas

In [ ]:
```
df_reducido =  df_airports[['type','name']]
df_reducido
```

In [ ]:
```
df_reducido
```

In [ ]:
```
df_reducido.rename(columns={'name': 'Nombre'})
```

In [ ]:
```
changes_columns = {'type':'tipo', 'name':'nombre'}

df_reducido.rename(columns=changes_columns)
```

Reemplazos en las celdas

In [ ]:
```
df_airports.region_name.unique()
```

In [ ]:
```
df_airports[df_airports.region_name.str.contains('Tierra del Fuego')]['re
```

In [ ]:
```
#df_airports.region_name.replace('Tierra del Fuego Province', 'Tierra del
```

In [ ]:
```
df_airports
```

In [ ]:
```
plt.bar(labels,values)
# Agregar etiquetas y título
plt.xlabel('Tipo de aeropuerto')
plt.ylabel('Cantidad')
plt.title('Cantidad de aeropuertos por tipo')

# Rotar etiquetas en el eje x para mejor legibilidad
plt.xticks(rotation=45)
# Mostrar el gráfico de barras
plt.show()
```

Uso de colores en el gráficos

In [ ]:
```
#plt.bar(labels, values, color='blue')  # Color azul
plt.bar(labels, values, color='#FF5733')  # Color naranja (código hexadec
plt.xlabel('Tipo de aeropuerto')
plt.ylabel('Cantidad')
plt.title('Cantidad de aeropuertos por tipo')

# Rotar etiquetas en el eje x para mejor legibilidad
plt.xticks(rotation=45)
# Mostrar el gráfico de barras
plt.show()
```

Tamaño de la figura

In [ ]:
```
plt.figure(figsize=(10, 6))
plt.bar(labels, values, color='#FF5733')  # Color naranja (código hexadec
plt.xlabel('Tipo de aeropuerto')
plt.ylabel('Cantidad')
plt.title('Cantidad de aeropuertos por tipo')

# Rotar etiquetas en el eje x para mejor legibilidad
plt.xticks(rotation=45)
# Mostrar el gráfico de barras
plt.show()
```

In [ ]:
```
df_airports
```

In [ ]:
```
df_airports.elevation_ft.min()
```